

Hayyuu AI SRS Executive Summary

Based on the polished source documents Hayyuu_AI_SRS_Polished.docx and Hayyuu_AI_SRS_Polished.md, this executive summary preserves the intent of the full Software Requirements Specification while condensing it for leadership review.

1. Product Overview

Hayyuu AI is an AI-native software engineering intelligence platform for individual software developers. In Version 1, it serves one authenticated user who can create and manage multiple isolated software projects. Each project acts as its own intelligence environment containing documents, conversations, requirements, tasks, memories, repository knowledge, and AI-generated insights.

The platform is intended to improve developer productivity by centralizing project knowledge, enabling AI-assisted reasoning, preserving decisions over time, and linking requirements, documents, code, and execution workflows in one system.

2. Scope of Version 1

Version 1 includes personal-project capabilities only. The user can upload documents, connect GitHub repositories, chat with AI, search project knowledge, extract and refine requirements, detect conflicts, generate and manage tasks, build persistent memory, inspect AI memory and knowledge, and operate durable AI workflows.

Version 1 explicitly excludes organizations, teams, invitations, shared workspaces, and role-based collaboration. However, the architecture must remain ready for future expansion into multi-user and multi-agent environments.

3. Users, Assumptions, and Constraints

The product is designed for software engineers, backend and frontend developers, AI engineers, students, freelancers, and independent developers. Users are expected to understand basic software engineering practices, but no prior AI expertise is required.

The specification assumes internet connectivity, configured AI model providers, and access to PostgreSQL, Redis, Qdrant, Neo4j, and GitHub API credentials. Version 1 is constrained to a single authenticated user with personal projects, personal memory, and personal knowledge assets only.

4. High-Level Architecture

Hayyuu AI is structured as a web application backed by a FastAPI service layer and several specialized intelligence subsystems. At a high level, the solution combines:

- A web application for user interaction
- A backend API layer for business logic and orchestration
- An AI runtime for planning, tool use, reflection, and workflow execution
- Project services for documents, repositories, requirements, tasks, memory, and knowledge
- A memory engine and knowledge engine for retrieval and structured understanding
- PostgreSQL for core application data
- Redis for caching and asynchronous coordination
- Qdrant for vector retrieval
- Neo4j for knowledge graph relationships

This architecture is intended to support explainable AI behavior, long-running workflow durability, retrieval-augmented generation, and future scaling.

5. Business Goals

The full SRS defines Hayyuu AI as a system that should centralize project knowledge, continuously learn from interactions, preserve persistent memory, understand both documentation and source repositories, connect requirements to implementation, improve developer productivity, explain AI reasoning, and preserve engineering decisions. A key strategic goal is to enable future evolution into a collaborative AI software engineering platform without replacing the core intelligence architecture.

6. Core Functional Capability Areas

The functional requirements in the full SRS can be summarized into the following major capability groups:

6.1 Platform Foundation

The platform must support secure account registration and login, user authentication, project creation and management, document upload and processing, notification handling, configuration management, and secure operation of all user data and system components.

6.2 AI Interaction and Project Understanding

Users must be able to converse with an AI assistant in the context of a specific project. The assistant is expected to reason over uploaded documents, repository content, project memory, structured knowledge, and retrieved context. The system must support project chat, retrieval-augmented responses, streaming output, and visibility into why the AI produced a given answer.

6.3 Persistent Memory and Knowledge

The platform distinguishes memory from knowledge. Memory captures experiences, interactions, and evolving context across working, short-term, long-term, episodic, and semantic forms. Knowledge captures structured facts derived from documents, repositories, conversations, requirements, and other project artifacts. Together, these systems allow the platform to learn over time and provide continuity across sessions.

6.4 Knowledge Graph and Search

The system must connect requirements, tasks, documents, technologies, repository artifacts, and other entities into a graph of relationships. It must also provide unified search across keyword, semantic, hybrid, and graph-based retrieval modes so that users and AI agents can quickly find relevant information across the project.

6.5 Requirements and Conflict Intelligence

The requirements engineering subsystem must convert unstructured project information into structured software requirements. The conflict detection subsystem must identify inconsistencies, ambiguities, contradictions, gaps, and other risks across requirements and project knowledge so that problems are found early and the project remains coherent.

6.6 Task and Delivery Intelligence

The task management subsystem must support creating, organizing, tracking, and analyzing development tasks linked to documents, conversations, requirements, and GitHub issues. This allows the product to bridge planning, implementation, and ongoing execution.

6.7 GitHub and Implementation Awareness

The GitHub integration subsystem must connect repositories, retrieve implementation signals, and help Hayyuu AI understand how a project is actually built. This supports stronger alignment between intended requirements and actual source code, issues, commits, and pull request activity.

6.8 Explainability, Inspection, and Agent Runtime

The platform includes an AI memory and knowledge inspector so users can understand what the system knows, what it remembers, and how answers were produced. Underneath, an AI agent runtime, AI tool system, and durable workflow system must support planning, execution, reflection, controlled tool use, background jobs, checkpointing, and recovery from failures or restarts.

7. Non-Functional Requirements

The SRS emphasizes that Hayyuu AI is not only feature-rich but also production-oriented.

- Performance: normal API requests should average under 500 ms; semantic search should return in under 2 seconds; hybrid search should return in under 3 seconds;

long-running work should execute asynchronously; and AI responses should stream progressively.

- Scalability: API instances and workers must scale independently, the architecture should support database indexing and caching strategies, and the platform should remain compatible with future multi-agent deployment.
- Reliability and durability: the system must recover from crashes, worker failures, network failures, and model failures. AI workflows must survive restarts, and critical operations must preserve data consistency.
- Availability: the production target is 99% availability, and failures in external services such as AI providers or GitHub must degrade gracefully rather than crash the platform.
- Maintainability: the codebase must remain modular, follow clean architecture principles, isolate business logic from frameworks and vendors, support dependency injection, and be documented across architecture, APIs, workflows, data, and deployment.
- Quality and observability: the system must include unit, integration, AI evaluation, and regression testing, plus structured logging, AI tracing, monitoring, and error tracking.
- Security: security is treated as foundational because the system manages private documents, source code, AI context, and derived knowledge. Authentication, authorization boundaries, secure storage, and safe access to tools and external systems are essential.

8. External Interfaces and Data Governance

The platform must integrate with multiple LLM providers through an abstraction layer rather than hard-coding a single vendor. It must also support embedding providers, official GitHub APIs, and service-layer access to PostgreSQL, Redis, Qdrant, and Neo4j.

From a data perspective, all project data belongs to the authenticated user. The system must govern creation, processing, storage, versioning, archiving, deletion, recovery, and future export of project artifacts. Deletion must remove records, embeddings, graph relationships, and stored files.

9. Version 1 Completion Criteria

Version 1 is considered complete when the core platform, AI capabilities, memory systems, knowledge systems, reliability mechanisms, and engineering quality expectations are all present. At a minimum, this includes authentication, project management, document upload, AI chat, retrieval-augmented generation, requirement extraction, conflict detection, task generation support, GitHub understanding, multiple memory types, knowledge extraction and search, a knowledge graph, background jobs, durable workflows, recovery mechanisms, testing, logging, monitoring, documentation, and containerized deployment.

10. Future Compatibility

Although Version 1 is single-user, the SRS requires the design to preserve a path to future organizations, team collaboration, permissions, shared projects, specialized autonomous agents, agent collaboration, and more advanced autonomous engineering capabilities such as code generation, pull request creation, automated testing, deployment assistance, and architecture optimization.

In short, Hayyu AI Version 1 is defined as a personal AI engineering assistant built on an architecture that can later evolve into a broader collaborative AI software engineering platform without major rework.